Preprint (v1)

How Many Sites, Participants, and Trials?

A Tutorial on Multilevel Power Analysis for Multisite Studies with Repeated Measures

Alicia Franco-Martínez[*1], **Miguel A. Vadillo**[2], and **Ricardo Rey-Sáez**[1]

[1] Departamento de Psicología Básica, Universidad Autónoma de Madrid

[2] Departamento de Psicología Básica I, Universidad Nacional de Educación a Distancia

*Correspondence: aliciafrancomartinez96@gmail.com

Abstract

Multisite collaborations are increasingly common in psychological science. Planning a multisite study with repeated measures involves more than deciding how many participants to recruit. Researchers must also determine how many sites to include and how many observations to collect from each participant. These decisions become particularly challenging when the experimental effect may vary across both participants and sites. Unfortunately, most widely used power analysis tools do not accommodate the complexity of these structures. In this tutorial, we provide a practical introduction to multilevel power analysis for three-level multisite studies. We first discuss how to identify and model the effect of interest, accounting for different sources of heterogeneity and the population to which researchers intend to generalize their findings. We then demonstrate how to estimate statistical power through simulation using `{simr}` and how to optimize the number of trials, participants, and sites using `{mlpwr}`, considering their relative costs. Finally, we illustrate how to explore alternative assumptions about effect size and between-site heterogeneity, verify the power of the selected architecture, assess estimation precision, and calculate the minimum detectable effect size. The accompanying R code provides an adaptable workflow to help researchers plan multisite studies that balance their inferential goals with their available resources.

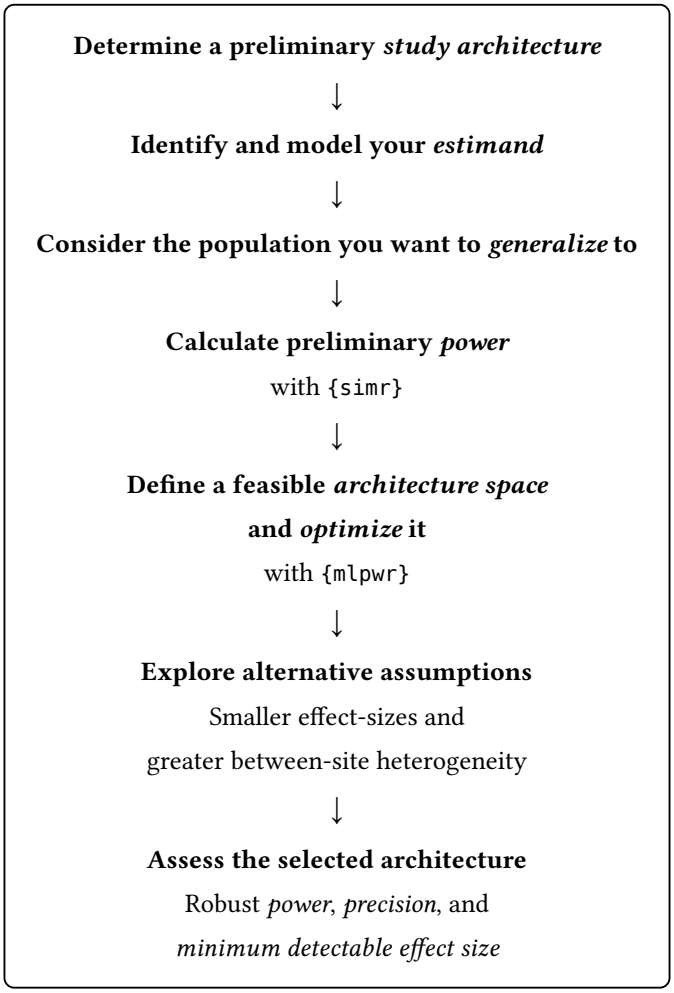
Keywords: multisite studies, statistical power, sample-size planning, repeated measures, multilevel models, simulation

Multisite collaborations are becoming increasingly common in psychological science. Perhaps you are considering coordinating one yourself. Brave choice! One of the keys to success will be careful planning. You might think that achieving sufficient statistical power is simply a matter of recruiting enough participants. However, in a multisite study with repeated measures, power also depends on how many sites you include and how many observations you collect from each participant. To refer to these decisions collectively, we will use the term *study architecture*.

Since multisite architectures are by definition hierarchical, multilevel models (also known as hierarchical or mixed-effects models) are a natural choice for analyzing these datasets. We assume basic familiarity with these models[1] and R, but no prior experience with simulation-based power analysis or `{simr}` (Green & MacLeod, 2016) and `{mlpwr}` (Zimmer et al., 2024) is needed. Figure 1 provides an overview of the workflow followed throughout this tutorial. You can follow all analyses in our openly available R script on Zenodo (doi.org/10.5281/zenodo.23415525).

Figure 1.

Workflow of the tutorial



[1] Readers new to these topics may consult Brysbaert & Debeer (2025), DeBruine & Barr (2021), and Singmann & Kellen (2020).

Determine a preliminary *study architecture*

You might not have heard the term "study architecture" before[2]. We use it to refer to the *hierarchy of levels* in the data and the number of units at each level (i.e., the *level sizes*). For instance, the multisite study we will use throughout this tutorial has three levels: the experimental task consists of 32 trials (Level 1) completed by each of 16 participants (Level 2) at each of 12 sites (Level 3). The hierarchy is therefore *trials* ⊂ *participants* ⊂ *sites*, and this particular architecture has a trial size of 32, a participant size of 16, and a site size of 12; in total, (32×16×12) 6,144 observations.

```
trials_size ← 32
partis_size ← 16
sites_size  ← 12

# Generate all combinations of trials, participants, and sites (6,144 rows)
architecture ← expand.grid(trial = 1:trials_size,
                           participant = factor(1:partis_size),
                           site = factor(1:sites_size))
```

Identify and model your *estimand*

Before thinking about power, you first need to decide which effect you want to estimate. Imagine your main research question concerns the effect of an experimental *condition* that varies from trial to trial within participants (such as the Stroop effect, which is the response time difference between congruent and incongruent trial conditions), hereafter coded as −0.5 and 0.5, respectively:

```
# Predictor (independent variable or factor): "condition"
condition_pattern ← rep(c(-0.5, 0.5), length.out = trials_size)

# Add the condition variable to the architecture design
architecture$condition ← rep(condition_pattern, times = partis_size * sites_size)
```

You want to know whether `condition` affects participants' `response` (e.g., response times). For the moment, let's ignore the hierarchical structure of your data—trials nested within participants—and start with the simplest possible model:

```
  response ~ condition
```

The coefficient for `condition` (i.e., the fixed slope) is your *estimand*: the average experimental effect you want to estimate. If the effect is as large as you expect, you want your study to have a good

[2]We introduce the term "study architecture" because other commonly used terms can be ambiguous in this context. We avoided the term "sample", because one could reasonably argue that the 12 sites themselves constitute a *sample of sites*, just as the trials could be regarded as sampled units. Also, in multisite research, the term *sample* is also commonly used to refer to the participant sample contributed by each site. Similarly, we avoided using "study design", because *design* is already widely used to describe how independent variables are manipulated or how an experimental task or procedure is structured.

chance of detecting it. Let's say, an 80% chance (i.e., 80% power). However, the model used to capture this estimand is currently too simplistic, because it assumes that the effect will be identical across participants and sites. Let's refine it little by little by accounting for different sources of heterogeneity in the data.

Of course, participants are unlikely to respond identically. Some may generally produce higher (or lower) responses than others, regardless of `condition`. You can represent this variation by adding a *participant-level random intercept*:

```
response ~ condition +        # Fixed effects
           (1 | participant)  # Random effects
```

This allows each participant to have their own average response level.

But participants may also differ in the size of the experimental effect itself. Some participants may be very sensitive to `condition`, others not at all, and some may even show an effect in the opposite direction. Such heterogeneity is common in psychological research, so unless there are strong reasons to assume otherwise, you should allow the effect of `condition` to vary across participants by adding a *participant-level random slope*:

```
response ~ condition +
           (1 + condition | participant)
```

This allows each participant to have their own average response level (the term `1`) and their own experimental effect (the term `condition`). In addition, the model estimates the covariance between both terms[3].

In a multisite study, the same logic applies to between-site heterogeneity, as experimental effects have been shown to vary across sites in previous multisite collaborations (e.g., Klein et al., 2018). From this point on, we will make the hierarchical structure explicit by writing participants as nested within sites: `site:participant`. This is essential when participant identifiers are repeated across sites (e.g., "P001", "P002"), because the model must distinguish "P001" at one site from "P001" at another. If participant identifiers are already unique across the entire dataset (e.g., "S1_P001", "S2_P001"), using participant alone would be equivalent. Still, the nested notation makes the three-level structure more transparent.

First, could sites differ in their average response levels? If so, you can add a *site-level random intercept*:

```
response ~ condition +
           (1 + condition | site:participant) +
           (1 | site)
```

[3]For instance, participants with higher mean performance might also be the ones who show larger experimental effects. You can set this covariance to zero using `||` instead of `|`.

This accounts for the possibility that some laboratories systematically produce higher or lower response values than others.

More relevant for many multisite studies, you should also ask whether the experimental effect itself could differ across sites. Perhaps the manipulation works more strongly in some laboratories than in others because of differences in recruitment, implementation, language, equipment, or other contextual factors. If you want your model to represent this possibility, you need a *site-level random slope*:

```
response ~ condition +
           (1 + condition | site:participant) +
           (1 + condition | site)
```

Now the model allows both participants and sites to differ not only in their average responses, but also in the size of the condition effect.

In this first example, the estimand is a *trial-level effect*, because `condition` varies across trials within participants. Similarly, you might instead be interested in a *participant-level effect*, such as the effect of `gender`, which varies across participants within sites. The model for this estimand would therefore be:

```
response ~ condition + gender +
           (1 + condition | site:participant) +
           (1 + condition + gender | site)
```

Here, the fixed effect of `gender` is the estimand: the average difference in response associated with `gender`. Note that `gender` is not included in the participant-level random-effects structure because it does not vary within participants. Therefore, you should not include a *participant-level random slope* for `gender`. However, because `gender` does vary across participants within sites, its effect can itself vary across sites. The *site-level random slope* for `gender` allows the model to represent this between-site heterogeneity.

So far, we have only considered the main effects of `condition` and `gender`. However, with multiple predictors, you may also be interested in whether their effects interact. For example, does the effect of `condition` differ across participants depending on their `gender`? If this interaction is your primary research interest, then your estimand would be the interaction between `condition` and `gender`, also known as a *cross-level interaction*:

```
response ~ condition * gender +
           (1 + condition | site:participant) +
           (1 + condition * gender | site)
```

Here, the *fixed effect* of `condition:gender` would be your estimand: how much the average effect of `condition` changes across levels of `gender`. Again, because `gender` does not vary within participants, neither `gender` nor the `condition×gender` interaction can have a participant-level

random slope. However, both can vary across sites, because sites contain participants from different levels of `gender` and trials from different levels of `condition`. The *site-level random slope* for `condition*gender` therefore allows the magnitude of this interaction to differ across sites.

And yes, your estimand could operate at an even higher level. Imagine that your multisite study includes several sites from each of two countries, and you are interested in the average difference between countries:

```
response ~ condition + country +
           (1 + condition | site:participant) +
           (1 + condition | site)
```

Here, the *fixed effect* of `country` is the estimand: the average difference in response between the two countries. Or you could model the interaction `condition*country` instead[4], as in the previous example. Because `country` does not vary within participants or within sites, it cannot have *participant-level* or *site-level random slopes*.

So far, all our estimands have been fixed effects, but heterogeneity itself could also be your target of inference. For instance, you might also be interested in the between-sites heterogeneity itself, so your estimand becomes the variance of the *site-level random intercepts* or *slopes* (see Bloom et al., 2017; Xie & Kelcey, 2026).

Consider the population you want to *generalize* to

Once you have identified your estimand, the next question is: to which population do you want to generalize it? In a multisite study, this depends on whether participants and sites are treated as *fixed* or *random* effects.

Let's assume again that your estimand is the trial-level effect of `condition`. Imagine that participants are treated as random, but the effect of `condition` is estimated separately for each of the particular site in your study:

```
response ~ condition * site +
           (1 + condition | site:participant)
```

The interaction `condition*site` allows the `condition` effect to differ across the recruited sites, but these site-specific effects are treated as *fixed*. In other words, the model makes no assumption that these sites represent a broader population of sites. Your inference can generalize across participants *similar to those sampled within these sites*, but the site-specific effects refer to the *particular sites that participated in the study* (not to *any potential site*).

This may be appropriate if those sites are themselves the units you care about (e.g., because something specific happened in those particular sites) or if you have recruited few sites. However,

[4] In R syntax, `condition*country` models both the interaction and the main effects of `condition` and `country`. If you'd prefer to model only the interaction, you can use `condition:country` instead.

multisite studies often aim to make broader inferences: we would like the estimated effect to generalize not only to new participants, but also to *new sites similar to those that were sampled*. To do that, the site-specific `condition` effects should instead be treated as *random*:

```
response ~ condition +
           (1 + condition | site:participant) +
           (1 + condition | site)
```

The same logic applies to participants. Treating participant-specific effects as fixed restricts inference to the particular participants observed, whereas treating them as random allows the model to regard them as a sample from a broader participant population.

You might be asking then: if random effects allow broader generalization, why would anyone model these effects as fixed? The reason is that broader generalization requires the model to acknowledge that the effect may genuinely vary across participants or sites. Once this heterogeneity is modeled, the average fixed effect is estimated with greater uncertainty than under a model that treats the observed units as fixed (Barr et al., 2013). Consequently, detecting the estimand may require more information, that is, larger trial, participant, or site sizes. Broader generalization therefore places stronger demands on the study architecture: you may need more trials, participants, and sites to detect the effect with sufficient power.

How actual multisite studies have modeled their architectures. Multisite studies tend to have a natural *trials* ⊂ *participants* ⊂ *sites* hierarchy. Some study architecture planning approaches explicitly represent this hierarchy and allow the experimental effect itself to vary across sites through *site-level random slopes*, for example, Adetula et al. (2026) or Coles et al. (2022); see also the pre-data Registered Report by Meltzoff et al. (2026). Other studies represent the three-level structure but account for between-site variability only through *site-level random intercepts*, assuming a common experimental effect across sites (e.g., Hall et al., 2024; Kosie et al., 2024; Visser et al., 2026). In other cases, potentially relevant sources of dependency are omitted from architecture planning: The site level may be ignored despite multisite data collection (e.g., Brohmer et al., 2024; Chen et al., 2025; Lo et al., 2024; The ManyBabies Consortium, 2020), or sites may instead be treated as separately powered replications (e.g., Dang et al., 2021; Nave et al., 2025; Wagenmakers et al., 2016). Likewise, repeated measures (trial-level) are sometimes aggregated at the participant level rather than explicitly modeled in prospective power calculations (e.g., The ManyBabies Consortium, 2020; Wagenmakers et al., 2016; Dang et al., 2021; Brohmer et al., 2024).

Calculate *power* for a preliminary architecture

Now that you have defined your preliminary architecture and identified your estimand, your next step is to estimate the power of this architecture to detect it. You may be familiar with other standard power software, such as G*Power (Faul et al., 2007) or the R package `{pwr}` (Champely, 2020). They are widely used to conduct power analyses for *t*-tests, ANOVAs, correlations, or linear

regression models, and rely on closed-form analytical solutions. Unfortunately, for a multilevel model like yours, such closed-form solutions do not exist. Instead, you have to estimate power through simulation:

a) Select population values for the model parameters,
b) simulate a sample of observations from this population with the chosen architecture,
c) fit the planned analysis model, and
d) test the estimand.

We then repeat the steps b) to d) many times. The proportion of simulated studies in which the null hypothesis is rejected is your estimated power. But let's first perform each operation separately before automating the whole procedure.

Your model can be represented conceptually as:

```
response = beta_intercept + beta_condition*condition +       # fixed effects
           u_parti_intercept + u_parti_condition*condition + # parti random effects
           u_site_intercept + u_site_condition*condition +   # site random effects
           epsilon                                           # residual error

# (*) The notation extends naturally to additional predictors
```

Where `beta` states for fixed effects and `u` states for the specific random effects of participant (`u_parti`) and site (`u_site`). The values of these parameters are *assumed*, so it is crucial that you justify them using prior evidence such as previous data, pilot or meta-analytic studies, plausible ranges… The Supplementary Material (available at doi.org/10.5281/zenodo.23416076) provides an example of how to do this using estimates from a previous single-site study. For the current illustration, let's assume that, in the population, the parameter values are:

```
# Fixed effects
beta_intercept ← 0        # fixed intercept
beta_condition ← 0.15     # fixed effect of "condition" (your estimand)

# Random effects (their standard deviations: sd)
sd_parti_intercept ← 0.50 # parti-level random intercept
sd_parti_condition ← 0.55 # parti-level random slope of "condition"

sd_site_intercept ← 0.20  # site-level random intercept
sd_site_condition ← 0.15  # site-level random slope of "condition"

# Residual error
sd_residual ← 1.00
```

Let's also assume zero covariance between the random effects, for the sake of simplicity:

```
# Variance-covariance matrix for parti-level random effects:
V_parti ← matrix(c(sd_parti_intercept^2, 0,
                   0, sd_parti_condition^2),
                 nrow = 2, byrow = TRUE)

# Variance-covariance matrix for site-level random effects:
V_site ← matrix(c(sd_site_intercept^2, 0,
                  0, sd_site_condition^2),
                nrow = 2, byrow = TRUE)
```

You are now ready to build your data-generating model with `makeLmer()`:

```
model ← simr::makeLmer(
  formula = response ~ condition +
                       (1 + condition | site:participant) +
                       (1 + condition | site),
  fixef = c(beta_intercept, beta_condition),
  VarCorr = list(V_parti, V_site),
  sigma = sd_residual,
  data = architecture
)
```

With your model ready, you can now simulate responses, fit the model to the simulated data, and test whether your estimand is statistically significant:

```
set.seed(0)  # For reproducibility
simulated_response ← simr::doSim(model)

fitted_model ← simr::doFit(simulated_response, model)

# Is the estimand detected in this replica?
p_value ← simr::doTest(fitted_model, simr::fixed("condition", "t"))

p_value < 0.05
#> [1] TRUE
```

This gives the result of one replication. To estimate power, you need to repeat the procedure many times and estimate the power as the proportion of replications in which the effect was significant. The function `powerSim()` from `{simr}` computes this automatically with `nsim` replications[5]:

[5]When running these simulations, you may encounter singular fits (boundary estimates) or convergence warnings, especially with few sites. Singular fits occur when random-effect variances are estimated as close to zero or correlations between random effects approach −1 or 1. Convergence warnings indicate difficulties estimating model parameters and resulting solutions might be inadequate. Both problems may arise when the data provide insufficient information to estimate a complex random-effects structure. If these problems occur in most replications, consider fixing to 0 the correlations between random effects (using `||` instead of `|`). Alternatively, you could use a Bayesian model (e.g., with `{brms}`) to regularize parameter estimates, although this is computationally more expensive. You can inspect the warnings using `power_preliminary$warnings`. Occasional warnings are not necessarily concerning, but frequent estimation problems should be investigated before interpreting the power estimates.

```
power_preliminary ← simr::powerSim(
  model,
  test = simr::fixed("condition", "t"),
  nsim = 1000, alpha = 0.05,
  seed = 0, progress = FALSE
)
#> Power for predictor 'condition', (95% confidence interval):
#>       55.80% (52.66, 58.91)
```

With these assumptions, this preliminary architecture has approximately 56% power. Since this is insufficient, you should increase the level sizes of your architecture. But... by how much? Should you increase the number of trials, participants, or sites? Calculating power with `powerSim()` for every possible combination would be computationally expensive. So, let's use `{mlpwr}` to find an optimal combination within a feasible *architecture space*.

Define a feasible *architecture space* and optimize it

You now know that your preliminary architecture provides insufficient power, so the next step is to consider how many units you could feasibly collect at each level. Let's imagine that you could increase the task to a maximum of 48 trials, require up to 36 participants per site, and recruit up to 20 sites. These limits define your *architecture space*:

```
architecture_space ← list(trials_size = c(32, 48),
                          partis_size = c(16, 36),
                          sites_size  = c(12, 20))
```

Instead of evaluating every possible combination, `find.design()` from `{mlpwr}` uses a surrogate model to search this space efficiently. This function requires you to create a `simfun` and a `costfun`. Let's see how:

Step I. Create the *simulation* function

The `simfun` should receive an architecture and return `TRUE` when the estimand is detected and `FALSE` otherwise. Fortunately, everything required inside this function is essentially what we have already learnt in the previous sections:

```
simfun_multisite ← function(trials_size, partis_size, sites_size) {

  # Create the architecture (same as before):
  architecture ← expand.grid(trial = 1:trials_size,
                             participant = factor(1:partis_size),
                             site = factor(1:sites_size))

  condition_pattern ← rep(c(-0.5, 0.5), length.out = trials_size)
  architecture$condition ← rep(condition_pattern, times = partis_size * sites_size)
```

```
  # Specify the model parameters (values already defined in the workspace):
  params <- list(beta = c(beta_intercept, beta_condition),
                 theta = c(sd_parti_intercept / sd_residual,
                           0,
                           sd_parti_condition / sd_residual,
                           sd_site_intercept / sd_residual,
                           0,
                           sd_site_condition / sd_residual),
                 sigma = sd_residual)

  # Simulate one dataset:
  response <- lme4::simulate.formula( ~ condition +
                                        (1 + condition | site:participant) +
                                        (1 + condition | site),
                                      newdata = architecture,
                                      newparams = params)
  architecture$response <- response[[1]] # Include the simulated response to data

  # Fit the model to the simulated dataset:
  fitted_model <- lmerTest::lmer(response ~ condition +
                                           (1 + condition | site:participant) +
                                           (1 + condition | site),
                                 data = architecture)

  # Test the estimand:
  p_value <- coef(summary(fitted_model))["condition", "Pr(>|t|)"]

  return(p_value < 0.05) # TRUE if detected
}
```

Step II. Create the *cost* function

Two architectures may achieve the same power but have substantially different costs: An architecture with 5 sites and 40 participants per site might achieve the same power as one with 20 sites with 10 participants each, but the latter could be considerably more expensive. How much more expensive? That's something that you can represent in your `costfun`. For example:

```
trial_cost <- 1    # Each trial has a cost of 1 unit.
parti_cost <- 25   # Each participant has an extra cost of 25 units.
site_cost  <- 1000 # Each site has an extra cost of 1000 units.
```

These values are arbitrary units reflecting how difficult or resource-intensive it is to increase each level of the architecture (in your specific research context). The `costfun` computes the total cost of a given architecture by summing the costs of all your trials, participants and sites:

```
costfun_multisite ← function(trials_size, partis_size, sites_size) {
  total_trials ← sites_size * partis_size * trials_size
  total_partis ← sites_size * partis_size
  total_sites  ← sites_size

  # Cost of all trials, all partis, and all sites:
  total_trial_cost ← total_trials * trial_cost
  total_parti_cost ← total_partis * parti_cost
  total_site_cost  ← total_sites * site_cost

  # Return: total cost of the architecture:
  return(total_trial_cost + total_parti_cost + total_site_cost)
}
```

Step III. Determine a desired power and optimize

```
desired_power ← 0.80 # let's say 80% power

set.seed(0)  # For reproducibility
optimal_architecture ← mlpwr::find.design(
  boundaries = architecture_space,
  simfun = simfun_multisite,
  costfun = costfun_multisite,
  power = desired_power,
  evaluations = 2000
)

summary(optimal_architecture)
#> Design: trials_size = 32, partis_size = 19, sites_size = 17
#>
#> Power: 0.80071,  SE: 0.01183,  Cost: 35411
#> Evaluations: 2006,  Time: 1626.67,  Updates: 47
#> Surrogate: Gaussian process regression

# Inspect the architectures evaluated by the surrogate model using:
mlpwr::simulations_data(optimal_architecture)

# (First rows only: 32 architectures in total; some columns omitted)
#>    trials_size partis_size sites_size  cost power_surrogate     power Evaluations
#> 1           32          16         12 22944       0.6440735 0.5588235          34
#> 2           42          32         12 37728       0.6951855 0.6176471          34
#> 3           38          20         14 31640       0.7059500 0.6176471          34
#> 4           45          21         16 39520       0.7750736 0.6470588          34
#> 5           32          17         15 29535       0.7251521 0.6764706          34
#> 6           32          21         16 35152       0.7855429 0.6862745         102
#>  ...
```

With the current assumptions, the optimal architecture consists of 32 trials, 19 participants per site, and 17 sites, with predicted power close to the desired .80.

This is a nice example where everything goes well, but sometimes `find.design()` may warn you that "`the predicted value consistently lies at the edge of the parameter space`". If that happens, it is worth checking whether the selected architecture reaches one of the boundaries of the architecture space you specified. Two things might be going on. If the predicted power is still below your desired level and the architecture lies at the upper boundary, you may need to increase the maximum level sizes. Conversely, if the selected architecture lies at the lower boundary while already achieving sufficient power, your search space may simply be broader than necessary, and you could consider allowing even smaller, cheaper architectures. In any case, if the optimization has not stabilized, you can also try increasing the number of evaluations.

Explore alternative assumptions

Your selected architecture depends on the assumed model parameters. You should therefore repeat the optimization under more demanding but plausible scenarios, such as smaller effect sizes or greater between-site heterogeneity, to assess how robust your selected architecture is.

```
# First, save the baseline assumptions:
baseline_beta_condition ← beta_condition
baseline_sd_site_condition ← sd_site_condition

# For a smaller effect:
beta_condition ← 0.10
sd_site_condition ← baseline_sd_site_condition

set.seed(0) # For reproducibility
smallerES_architecture ← mlpwr::find.design(
  boundaries = architecture_space,
  simfun = simfun_multisite,
  costfun = costfun_multisite,
  power = desired_power,
  evaluations = 2000
)

# For greater between-site heterogeneity:
beta_condition ← baseline_beta_condition
sd_site_condition ← 0.25

set.seed(0) # For reproducibility
greaterBSH_architecture ← mlpwr::find.design(
  boundaries = architecture_space,
  simfun = simfun_multisite,
  costfun = costfun_multisite,
  power = desired_power,
  evaluations = 2000
)
```

```
# Compare the three scenarios side by side:
scenario_comparison ← transform(
  rbind(baseline   = optimal_architecture$final$design,
        smallerES  = smallerES_architecture$final$design,
        greaterBSH = greaterBSH_architecture$final$design),
  total_cost = costfun_multisite(trials_size, partis_size, sites_size),
  power = round(c(optimal_architecture$final$power,
                  smallerES_architecture$final$power,
                  greaterBSH_architecture$final$power), 2))


scenario_comparison
#>            trials_size partis_size sites_size total_cost power
#> baseline            32          19         17      35411  0.80
#> smallerES           48          36         18      65304  0.59
#> greaterBSH          48          36         18      65304  0.63
```

In this example, neither alternative scenario achieves the desired power, suggesting that your architecture space needs to be expanded to accommodate more demanding scenarios. Note that a smaller effect size would require more information overall, while greater between-site heterogeneity would particularly require additional sites. More information is described in our script at doi.org/10.5281/zenodo.23415525.

In both alternative scenarios, the selected architectures approach the upper limits of the search space but still fall well below 80% power. A smaller effect requires more information overall, whereas greater between-site heterogeneity particularly increases the need for more sites. Therefore, your architecture space needs to be expanded if you'd like to accommodate more demanding scenarios.

Assess the selected architecture

Your optimization has identified an architecture expected to achieve the desired power at the lowest cost. However, to be efficient, `{mlpwr}` estimates power for only some architectures and uses these estimates to fit a surrogate model, which then predicts the power of all architectures in the space—without simulating each one extensively. For this reason, the power of the selected architecture should be re-estimated afterwards using a larger number of simulations. Additionally you can assess the precision with which your selected architecture can estimate your estimand and determine the smallest effect it can detect with a desired power.

Robust estimation of power (and precision)

First, extract the architecture selected by `{mlpwr}` and recreate the dataset structure. We also restore our baseline assumptions, which we modified in the previous section.

```r
# Restore baseline assumptions:
beta_condition ← baseline_beta_condition
sd_site_condition ← baseline_sd_site_condition

# Rewrite the architecture with the one selected by {mlpwr}:
trials_size ← optimal_architecture$final$design[["trials_size"]]
partis_size ← optimal_architecture$final$design[["partis_size"]]
sites_size  ← optimal_architecture$final$design[["sites_size"]]


# Create the selected architecture (same as before):
architecture ← expand.grid(trial = 1:trials_size,
                           participant = factor(1:partis_size),
                           site = factor(1:sites_size))

condition_pattern ← rep(c(-0.5, 0.5), length.out = trials_size)

architecture$condition ← rep(condition_pattern,
                             times = partis_size * sites_size)
```

Since we are already simulating the data, we can assess both power and precision at the same time. In each replication, we simulate a dataset, fit the planned analysis model, and extract, not only the *p*-value, but also the estimated effect, its standard error, and degrees of freedom.[6]

```r
# Redefine the model with the selected architecture:
selected_model ← simr::makeLmer(
  formula = response ~ condition +
    (1 + condition | site:participant) +
    (1 + condition | site),
  fixef = c(beta_intercept, beta_condition),
  VarCorr = list(V_parti, V_site),
  sigma = sd_residual,
  data = architecture
)

# Store the results of 1,000 simulated studies:
nsim_final ← 1000

p_values  ← rep(NA, nsim_final)
estimates ← rep(NA, nsim_final)
ses       ← rep(NA, nsim_final)
dfs       ← rep(NA, nsim_final)

set.seed(0) # For reproducibility
```

[6]We cannot use `simr::powerSim()` here because it does not directly return the individual estimates, standard errors, and degrees of freedom needed for our precision analysis.

```
for (i in 1:nsim_final) {
    # Simulate and fit the model:
    simulated_response <- simr::doSim(selected_model)
    fitted_model <- simr::doFit(simulated_response, selected_model)

    # Extract the results for your estimand:
    coefficient_table <- coef(summary(lmerTest::as_lmerModLmerTest(fitted_model)))

    p_values[i]  <- coefficient_table["condition", "Pr(>|t|)"]
    estimates[i] <- coefficient_table["condition", "Estimate"]
    ses[i]       <- coefficient_table["condition", "Std. Error"]
    dfs[i]       <- coefficient_table["condition", "df"]
}
```

Again, calculate power as the proportion of successful simulations in which the null hypothesis was rejected:

```
power_final <- mean(p_values < 0.05, na.rm = TRUE)
power_final
#> [1] 0.763
```

With 1,000 simulations, the estimated power is .763, somewhat below the .80 predicted by `mlpwr`. If detecting the originally assumed effect with 80% power remains your goal, you should reconsider the selected architecture, for example, by repeating the optimization with more evaluations or expanding the architecture space, and reassess the resulting design.

For precision, you can examine the variability of the estimated effects (called *empirical standard error*) and the average width of their 95% confidence intervals[7]. You can also calculate *empirical coverage*: the proportion of intervals containing the true effect used to generate the data (which should be close to 1-α under correct model specification).

```
# Assess precision and confidence interval coverage:
t_critical <- stats::qt(0.975, df = dfs)

ci_lower <- estimates - t_critical * ses
ci_upper <- estimates + t_critical * ses

precision_results <- data.frame(
  mean_estimate = mean(estimates, na.rm = TRUE),
  empirical_se = sd(estimates, na.rm = TRUE),
  mean_model_se = mean(ses, na.rm = TRUE),
  mean_95CI_width = mean(ci_upper - ci_lower, na.rm = TRUE),
  empirical_coverage = mean(
    ci_lower ⩽ beta_condition & ci_upper ⩾ beta_condition,
    na.rm = TRUE))
```

[7]For consistency with our significance test, the confidence intervals use the t distribution with Satterthwaite degrees of freedom.

```
precision_results
#>   mean_estimate empirical_se mean_model_se mean_95CI_width  empirical_coverage
#>       0.1484726   0.05359122    0.05116042       0.2162517               0.948
```

The empirical standard error is .054, the average confidence interval width is .216, and empirical coverage is 94.8%. While coverage assesses the calibration of the confidence intervals, their width indicates how precisely you can estimate the effect.

Calculate the minimum detectable effect size

Finally, you can calculate the *minimum detectable effect size* (MDES): the smallest true effect that your selected architecture can detect with the desired power, given your other model assumptions.

To approximate the MDES, you can progressively change the assumed effect size in increments of, let's say, .01 and recalculate power. If your architecture already achieves the desired power, you should search for smaller effects; otherwise, search for larger ones.

```
# Define the search:
effect_step ← 0.01
nsim_mde ← 200 # (!) More simulations should be used for the final MDES.

current_effect ← beta_condition
current_power ← power_final

mdes_results ← data.frame(effect_size = current_effect,
                          estimated_power = current_power)

# Determine the direction of the search:
search_down ← current_power ⩾ desired_power
direction ← if (search_down) -1 else 1

max_iter ← 50  # Maximum number of effect sizes to test
iter ← 0       # Counter for the number of iterations

repeat {
  iter ← iter + 1

  # Move to the next candidate effect size:
  current_effect ← current_effect + direction * effect_step

  # Build the model using the current candidate effect size:
  mdes_model ← simr::makeLmer(
    response ~ condition +
      (1 + condition | site:participant) +
      (1 + condition | site),
    fixef = c(beta_intercept, current_effect),
    VarCorr = list(V_parti, V_site),
    sigma = sd_residual,
    data = architecture)
```

```
  # Estimate power for the current effect size:
  mdes_result <- simr::powerSim(mdes_model,
                                test = simr::fixed("condition", "t"),
                                nsim = nsim_mdes, alpha = 0.05,
                                seed = 0, progress = FALSE)

  # Compute the proportion of simulations in which the effect was detected:
  current_power <- mdes_result$x / mdes_result$n

  # Store the effect size tested and its estimated power:
  mdes_results <- rbind(
    mdes_results,
    data.frame(effect_size = current_effect,
               estimated_power = current_power)
  )

  # Stop once the search crosses the desired power:
  if (search_down && current_power < desired_power) break
  if (!search_down && current_power >= desired_power) break

  # Stop if the maximum number of iterations is reached:
  if (iter >= max_iter) {
    warning(
      "Maximum number of iterations reached. ",
      "Last effect tested: ", round(current_effect, 3),
      "; estimated power: ", round(current_power, 3), "."
    )
    break
  }
}

mdes  #> [1] 0.16
```

In this example, the architecture achieves approximately 76% power for an effect of .15, and 83.5% for an effect of .16. Its approximate MDES is therefore .16. Because this search uses only 200 simulations per additional effect size, the result is preliminary. More simulations, particularly around the estimated threshold, would be needed for a robust final assessment.

Congratulations!

You have made it through one of the most technical parts of planning your multisite study. Hopefully, you now have a clearer answer to our initial question: How many sites, participants, and trials? Well, it depends! As we have seen, the answer depends on what you want to estimate, the population you want to generalize to, your assumptions about the data, and, of course, your available resources. There is no universally optimal architecture, but we hope this tutorial and its accompanying code will help you find one that works for your study. Good luck with your multisite adventure!

References

Adetula, A., Forscher, P. S., Basnight-Brown, D., Wagge, J. R., Kaliyapa, W. T., Polycarp, C. G., Malingumu, W. S., Azouaghe, S., Sambu, L., Ndukaihe, I. L. G., Adetula, G. A., Charyate, A., Ogbonnaya, C. E., Arinze, N. C., Shumiye, O. P., Jack, D. B., Ouoba, N. M., Bada, B. V., Khaoudi, A., … IJzerman, H. (2026). The Evaluation of Harm and Purity Transgressions in Africans: A Paradigmatic Replication of Rottman and Young (2019). *Advances in Methods and Practices in Psychological Science*, *9*(2). https://doi.org/10.1177/25152459261428031

Barr, D. J., Levy, R., Scheepers, C., & Tily, H. J. (2013). Random effects structure for confirmatory hypothesis testing: Keep it maximal. *Journal of Memory and Language*, *68*(3), 255–278. https://doi.org/10.1016/j.jml.2012.11.001

Bloom, H. S., Raudenbush, S. W., Weiss, M. J., & Porter, K. (2017). Using Multisite Experiments to Study Cross-Site Variation in Treatment Effects: A Hybrid Approach With Fixed Intercepts and a Random Treatment Coefficient. *Journal of Research on Educational Effectiveness*, *10*(4), 817–842. https://doi.org/10.1080/19345747.2016.1264518

Brohmer, H., Hofer, G., Bauch, S. A., Beitner, J., Berkessel, J. B., Corcoran, K., Garcia, D., Gruber, F. M., Giuliani, F., Jauk, E., Krammer, G., Malkoc, S., Metzler, H., Mües, H. M., Otto, K., Rahal, R.-M., Salwender, M., Sczesny, S., Stahlberg, D., … Athenstaedt, U. (2024). Effects of the Generic Masculine and Its Alternatives in Germanophone Countries: A Multi-Lab Replication and Extension of Stahlberg, Sczesny, and Braun (2001). *International Review of Social Psychology*, *37*(1), 17. https://doi.org/10.5334/irsp.522

Brysbaert, M., & Debeer, D. (2025). How to Run Linear Mixed Effects Analysis for Pairwise Comparisons? A Tutorial and a Proposal for the Calculation of Standardized Effect Sizes. *Journal of Cognition*, *8*(1), 5. https://doi.org/10.5334/joc.409

Champely, S. (2020). *pwr: Basic Functions for Power Analysis*. https://doi.org/10.32614/CRAN.package.pwr

Chen, S.-C., Buchanan, E. M., Kekecs, Z., Miller, J. K., Szabelska, A., Aczel, B., Bernabeu, P., Forscher, P. S., Szuts, A., Vally, Z., Al-Hoorie, A. H., Helmy, M., Silva, C. S. Alves da, Silva, L. Oliveira da, Moraes, Y. L. de, Santos Hsu, R. M. C., Looman Mafra, A., Valentova, J. V., Correa Varella, M. A., … Chartier, C. R. (2025). Investigating Object Orientation Effects Across 18 Languages. *Current Psychology*, *44*(24), 19148–19174. https://doi.org/10.1007/s12144-025-08304-x

Coles, N. A., March, D. S., Marmolejo-Ramos, F., Larsen, J. T., Arinze, N. C., Ndukaihe, I. L. G., Willis, M. L., Foroni, F., Reggev, N., Mokady, A., Forscher, P. S., Hunter, J. F., Kaminski, G., Yüvrük, E., Kapucu, A., Nagy, T., Hajdu, N., Tejada, J., Freitag, R. M. K., … Liuzza, M. T. (2022). A Multi-Lab Test of the Facial Feedback Hypothesis by the Many Smiles Collaboration. *Nature Human Behaviour*, *6*(12), 1731–1742. https://doi.org/10.1038/s41562-022-01458-9

Dang, J., Barker, P., Baumert, A., Bentvelzen, M., Berkman, E., Buchholz, N., Buczny, J., Chen, Z., De Cristofaro, V., Vries, L. de, Dewitte, S., Giacomantonio, M., Gong, R., Homan, M., Imhoff, R.,

Ismail, I., Jia, L., Kubiak, T., Lange, F., … Zinkernagel, A. (2021). A Multilab Replication of the Ego Depletion Effect. *Social Psychological and Personality Science*, *12*(1), 14–24. https://doi.org/10.1177/1948550619887702

DeBruine, L. M., & Barr, D. J. (2021). Understanding Mixed-Effects Models Through Data Simulation. *Advances in Methods and Practices in Psychological Science*, *4*(1). https://doi.org/10.1177/2515245920965119

Faul, F., Erdfelder, E., Lang, A.-G., & Buchner, A. (2007). G*Power 3: A flexible statistical power analysis program for the social, behavioral, and biomedical sciences. *Behavior Research Methods*, *39*(2), 175–191. https://doi.org/10.3758/BF03193146

Green, P., & MacLeod, C. J. (2016). simr: An R Package for Power Analysis of Generalised Linear Mixed Models by Simulation. *Methods in Ecology and Evolution*, *7*(4), 493–498. https://doi.org/10.1111/2041-210X.12504

Hall, B., Schmidt, K., Wagge, J., Lewis, S. C., Weissgerber, S. C., Kiunke, F., Pfuhl, G., Stieger, S., Tran, U. S., Barzykowski, K., Bogatyreva, N., Kowal, M., Massar, K., Pernerstofer, F., Sorokowski, P., Voracek, M., Chartier, C. R., Brandt, M. J., Grahe, J. E., … Buchanan, E. M. (2024). Registered Replication Report: A Large Multilab Cross-Cultural Conceptual Replication of Turri et al. (2015). *Advances in Methods and Practices in Psychological Science*, *7*(4), 1–38. https://doi.org/10.1177/25152459241267902

Klein, R. A., Vianello, M., Hasselman, F., Adams, B. G., Adams, R. B., Alper, S., Aveyard, M., Axt, J., Babalola, M. T., Bahník, Š., Berkics, M., Bernstein, M. J., Berry, D. R., Bialobrzeska, O., Bocian, K., Brandt, M., Busching, R., Cai, H., Cambier, F., … Chatard, A. (2018). Many Labs 2: Investigating Variation in Replicability Across Samples and Settings. *Advances in Methods and Practices in Psychological Science*, *1*(4), 443–490. https://doi.org/10.1177/2515245918810225

Kosie, J. E., Zettersten, M., Abu-Zhaya, R., Amso, D., Babineau, M., Baumgartner, H. A., Bazhydai, M., Belia, M., Benavides-Varela, S., Bergmann, C., Berteletti, I., Black, A. K., Borba Borges, P., Borovsky, A., Byers-Heinlein, K., Cabrera, L., Calignano, G., Cao, A., Cox, C. M. M., … Lew-Williams, C. (2024,). *ManyBabies 5: A Large-Scale Investigation of the Proposed Shift From Familiarity Preference to Novelty Preference in Infant Looking Time*. PsyArXiv. https://doi.org/10.31234/osf.io/ck3vd

Lo, C. Y., Zendel, B. R., Baskent, D., Boyle, C., Coffey, E., Gagne, N., Habibi, A., Harding, E., Keijzer, M., & Kreutz, G. (2024). Speech-in-Noise, Psychosocial, and Heart Rate Variability Outcomes of Group Singing or Audiobook Club Interventions for Older Adults With Unaddressed Hearing Loss: A SingWell Project Multisite, Randomized Controlled Trial, Registered Report Protocol. *PLOS ONE*, *19*(12), e0314473. https://doi.org/10.1371/journal.pone.0314473

The ManyBabies Consortium. (2020). Quantifying Sources of Variability in Infancy Research Using the Infant-Directed-Speech Preference. *Advances in Methods and Practices in Psychological Science*, *3*(1), 24–52. https://doi.org/10.1177/2515245919900809

Meltzoff, A. N., Farwaha, S., Slaughter, V., Simpson, E. A., Brooks, R., Nielsen, M., Baumgartner, H. A., Bayet, L., Benavides-Varela, S., Boyce, V., Byers-Heinlein, K., Dönicke, J., Flaherty, B. P., Forbes, S. H., Frank, M. C., Geraci, A., Hamlin, J. K., Heimann, M., Keven, N., … Zeng, G. (2026,). *ManyBabies 6: A Multi-Lab Investigation of Neonatal and Early Imitation*. PsyArXiv. https://doi.org/10.31234/osf.io/fe7pj_v1

Nave, K. M., Hannon, E. E., & Snyder, J. S. (2025). Registered Report: Replication and Extension of Nozaradan, Peretz, Missal and Mouraux (2011). *Biorxiv*. https://doi.org/10.1101/2025.03.13.643168

Singmann, H., & Kellen, D. (2020). An introduction to mixed models for experimental psychology. In D. H. Spieler & E. H. Schumacher (Eds.), *New methods in cognitive psychology: New methods in cognitive psychology* (pp. 4–31). Routledge. https://doi.org/10.4324/9780429318405-2

Visser, I., Geambaşu, A., Kachergis, G., Oliveira, C. M., Rocha-Hidalgo, J., Zettersten, M., Ahamat, T., Alessandroni, N., Althaus, N., Arunachalam, S., Aussems, S., Axelsson, E. L., Aydin, Z., Baumgartner, H. A., Bergmann, C., Bettoni, R., Black, A. K., Bulf, H., Byers-Heinlein, K., … Levelt, C. (2026). ManyBabies 3: A Multi-Lab Study of Infant Algebraic Rule Learning. *Developmental Science*, *29*(6), e70259. https://doi.org/10.1111/desc.70259

Wagenmakers, E.-J., Beek, T., Dijkhoff, L., Gronau, Q. F., Acosta, A., Adams, R. B., Jr., Albohn, D. N., Allard, E. S., Benning, S. D., Blouin-Hudon, E.-M., Bulnes, L. C., Caldwell, T. L., Calin-Jageman, R. J., Capaldi, C. A., Carfagno, N. S., Chasten, K. T., Cleeremans, A., Connell, L., DeCicco, J. M., … Zwaan, R. A. (2016). Registered Replication Report: Strack, Martin, & Stepper (1988). *Perspectives on Psychological Science*, *11*(6), 917–928. https://doi.org/10.1177/1745691616674458

Xie, Y., & Kelcey, B. (2026). Multisite Partially Nested Designs: Estimation, Inference, and Design. *Journal of Research on Educational Effectiveness*, *19*(3), 728–753. https://doi.org/10.1080/19345747.2025.2567241

Zimmer, F., Henninger, M., & Debelak, R. (2024). Sample Size Planning for Complex Study Designs: A Tutorial for the mlpwr Package. *Behavior Research Methods*, *56*, 5246–5263. https://doi.org/10.3758/s13428-023-02269-0